

- Derivation of quasipotential Bethe-Salpeter equation
 - Quasipotential approximation
 - Partial-wave expansion
 - Fixed parity
- Solution of quasipotential Bethe-Salpeter equation
 - Independent amplitudes
 - Treatment of the singularity
 - Transformation to a matrix equation
 - For old code
 - Pole search
- 2-2 cross section
 - The cross section for the channel considered
 - Argand plot
- Three body decay
 - kinematics
 - Lorentz boost
 - Amplitude
 - Decay width
- qBSE package
 - Data Structures for the Interactions
 - `structSys`
 - `structInterAction`
 - Data Structures for the Dimensions
 - `structChannel`
 - `structIndependentHelicity`
 - Additional Data Structures
 - `structMomentum`
 - `structHelicity`
 - Particle
 - `particles!(filename::String)`
 - Functions for the qBSE
 - `function preprocessing(Sys, qn, channels, Ff, config, Np, Nx, Nphi)`
 - `function FFre(k, cutoffi, cutofff; cutoff_re_type=:Lambda, CHi=nothing, CHf=nothing, key_ex=0)`
 - `function propFFex(k, key_ex, cutoff; cutoff_ex_type=:Lambda, FF_ex_type=3)`
 - `function fV(k, l, SYS, IA0, CHf, CHi, VVertex)`
 - Functions for rescattering amplitudes and poles
 - `auxiliary function`
 - `function resc0(Range, iER, qn, SYS, IA, CH, IH, VVertex)`
 - `function resc(Sys, qn, Range, channels, Ff, cutoff, VVertex; Np=10, Nx=10, Nphi=5)`
 - `function simpleXsection(ER, resM2, CH, qn; Ep="cm",)`
 - `function lambda(m1, m2, m3)`
 - Decay
 - `function proc(pf, pin, amps)`
 - `function LorentzBoost(k::SVector{5,Float64}, p::SVector{5,Float64})`
 - `function setTGA(par, sij, k, tecm, i, j)`
 - `TGA(para, cfinal, cinter, ranges)`
 - `function Vertex14(k, P, l, Vert)`

- Quasipotential approximation
 - Partial-wave expansion
 - Fixed parity
 - Transformation to a matrix equation
 - Code
 - Pole search
 - Physical observable
 - The cross section for the channel considered
 - Argand plot
- Three body decay
 - kinematics
 - Lorentz boost
 - Amplitude
 - Decay width
 - The case with one rescattering
 - Phase space
 - Differential decay width
 - The case with more than one rescattering
- qBSE package
 - Data Structures for the Interactions
 - `structSys`
 - `structInterAction`
 - Data Structures for the Dimensions
 - `structChannel`
 - `structIndependentHelicity`
 - Additional Data Structures
 - `structMomentum`
 - `structHelicity`
 - Particle
 - `structParticle`
 - function to read particle data file
 - Functions for the qBSE
 - `function preprocessing(Sys, qn, channels, Ff, cutoff, Np, Nx, Nphi)`
 - `function FFre(k, cutoffi, cutofff; cutoff_re_type=:Lambda, CHi=nothing, CHf=nothing, key_ex=0)`
 - `function propFFex(k, key_ex, cutoff; cutoff_ex_type=:Lambda, FF_ex_type=3)`
 - `function fV(k, l, SYS, IA0, CHf, CHi, VVertex)`
 - Functions for rescattering amplitudes and poles
 - auxiliary function
 - `function resc0(Range, iER, qn, SYS, IA, CH, IH, VVertex; eps)`
 - `function resc(Sys, qn, Range, channels, Ff, cutoff, VVertex; Np=10, Nx=10, Nphi=5, eps=+1e-4im)`
 - `function simpleXsection(ER, resM2, CH, qn; Ep=("cm",))`
 - `function lambda(m1, m2, m3)`
 - Decay
 - `function proc(pf, pin, amps)`
 - `function LorentzBoostRotation(k, tecm, p1, p2)`
 - `function setTGA(par, sij, k, tecm, i, j)`
 - `TGA(para, cfinal, cinter, ranges)`
 - `function Vertex14(k, P, l, Vert)`

Derivation of quasipotential Bethe-Salpeter equation

Quasipotential approximation

The general form of the Bethe-Salpeter equation (BSE) for the scattering amplitude can be written as follows:

$$\mathcal{M}(k'_1 k'_2, k_1 k_2; P) = \mathcal{V}(k'_1 k'_2, k_1 k_2; P) + \int \frac{d^4 k''_2}{(2\pi)^4} \mathcal{V}(k'_1 k'_2, k''_1 k''_2; P) G(k''_1 k''_2; P) \mathcal{M}(k''_1 k''_2, k_1 k_2; P),$$

where $\mathcal{V}$ is the potential kernel and G is the propagator for the two constituent particles. The total momentum of the system is denoted by $P = k_1 + k_2 = k'_1 + k'_2 = k''_1 + k''_2$, with $k_{1,2}$, $k'_{1,2}$ and $k''_{1,2}$ being the initial, final and intermediate momenta, respectively.

The Bethe-Salpeter equation can be succinctly expressed as

$$\mathcal{M} = \mathcal{V} + \mathcal{V} G \mathcal{M},$$

The Bethe-Salpeter (BS) equation for the amputated scattering matrix without external legs, denoted as $\mathcal{M}_{\mu'_1 \mu'_2 \mu_1 \mu_2}$, is given by:

$$\mathcal{M}_{\mu'_1 \mu'_2 \mu_1 \mu_2} = \mathcal{V}_{\mu'_1 \mu'_2 \mu_1 \mu_2} + \mathcal{V}_{\mu'_1 \mu'_2 \rho_1 \rho_2} G^{\rho_1 \rho_2 \rho'_1 \rho'_2} \mathcal{M}_{\rho'_1 \rho'_2 \mu_1 \mu_2}, \quad (2)$$

where the propagator for the two constituent particles is given by

$$G^{\rho_1 \rho_2 \rho'_1 \rho'_2} = G^{\rho'_1 \rho_1}_1 \otimes G^{\rho'_2 \rho_2}_2 = \frac{i P_1^{\rho'_1 \rho_1}}{(k_1^2 - m_1^2)} \otimes \frac{i P_2^{\rho'_2 \rho_2}}{(k_2^2 - m_2^2)} = [\sum_{\lambda_1} A_{1\lambda_1}^{\rho'_1} \bar{A}_{1\lambda_1}^{\rho_1} \otimes \sum_{\lambda} A_{2\lambda}^{\rho'_2} \bar{A}_{2\lambda}^{\rho_2}] \tilde{G}_0, \quad (3)$$

where $P_i^{\rho'_i \rho_i}$ is a general tensor structure. For example, for two vector mesons, $P^{\rho' \rho} = (-g^{\rho' \rho} + k^{\rho'} k^{\rho} / m^2) = \sum_{\lambda} \varepsilon_{\lambda}^{\rho'} \varepsilon_{\lambda}^{\rho*}$; for two spin-1/2 baryons, $P = (\gamma \cdot k + m) = 2m \sum_{\lambda} u_{\lambda} \bar{u}_{\lambda}$ (Here we adopt convention $\bar{u}u = 1$). In general, such propagators are difficult to handle because the potential $\mathcal{V}$ and amplitude $\mathcal{M}$ cannot be factorized. However, since the dominant contribution comes from the region where both constituents are near their mass shells, a form factor or cutoff is usually introduced to restrict the propagator to the near on-shell region. Therefore, it is reasonable to approximate $P^{\rho' \rho}$ by its on-shell value, which can be expressed as a sum over polarization vectors or spinors: $P^{\rho' \rho} \approx \sum_{\lambda} A_{\lambda}^{\rho'} \bar{A}_{\lambda}^{\rho}$, where A denotes the polarization vector ε or spinor u . The factor $\tilde{G}_0$ is given by $\tilde{G}_0 = -\frac{1}{(k_1^2 - m_1^2)(k_2^2 - m_2^2)}$, where, for **fermions**, an additional $2m$ factor should be included, as required by the spinor convention.

After projecting the Bethe-Salpeter (BS) equation onto the polarization states, i.e., multiplying both sides by $A_{1\lambda_1} A_{2\lambda_2}$ and $\bar{A}_{1\lambda'_1} \bar{A}_{2\lambda'_2}$, the BS equation becomes

$$i \bar{A}_{1\lambda'_1} \bar{A}_{2\lambda'_2} \mathcal{M} A_{1\lambda_1} A_{2\lambda_2} = i \bar{A}_{1\lambda'_1} \bar{A}_{2\lambda'_2} \mathcal{V} A_{1\lambda_1} A_{2\lambda_2} + \sum_{\lambda'_1, \lambda'_2} (i \bar{A}_{1\lambda'_1} \bar{A}_{2\lambda'_2} \mathcal{V} A_{\lambda'_1} A_{\lambda'_2}) \tilde{G}_0 / i (i \bar{A}_{\lambda'_1} \bar{A}_{\lambda'_2} \mathcal{M} A_{1\lambda_1} A_{2\lambda_2}),$$

We define $\bar{A}_{1\lambda'_1} \bar{A}_{2\lambda'_2} \mathcal{M} A_{1\lambda_1} A_{2\lambda_2}$ as $\mathcal{M}_{\lambda'_1 \lambda'_2 \lambda_1 \lambda_2}$, and similarly define the potential as $\bar{A}_{1\lambda'_1} \bar{A}_{2\lambda'_2} \mathcal{V} A_{1\lambda_1} A_{2\lambda_2} \equiv \mathcal{V}_{\lambda'_1 \lambda'_2 \lambda_1 \lambda_2}$. In addition, we include the form factors of the two interacting particles in both the potential and the amplitude; for example, for the potential, $f(k'_1) f(k'_2) \mathcal{V}_{\lambda'_1 \lambda'_2 \lambda_1 \lambda_2} f(k_1) f(k_2)$. Hereafter, we will not explicitly show such form factors unless necessary. This means that the obtained amplitudes include the form factor when the particles are off-shell.

The Gross form of proposed quasipotential propagators for particles 1 and 2 with mass m_1 and m_2 written down in the center of mass frame where $P = (W, \mathbf{0})$ with particle 2 being on shell are

$$\tilde{G}_0 = \frac{-1}{(k_1^2 - m_1^2)(k_2^2 - m_2^2)} \rightarrow g = 2\pi i \frac{\delta^+(k_2^2 - m_2^2)}{k_1^2 - m_1^2} = 2\pi i \frac{\delta(k_2^0 - E_2)}{2E_2[(W - E_2)^2 - E_1^2]}, \quad (5)$$

where $k_1 = (k_1^0, \mathbf{k}) = (E_1, \mathbf{k})$, $k_2 = (k_2^0, -\mathbf{k}) = (W - E_1, -\mathbf{k})$ with $E_1 = \sqrt{m_1^2 + |\mathbf{k}|^2}$.

With the define of $G_0 = g/(2\pi i) = \frac{1}{2E_2[(W - E_2)^2 - E_1^2]}$, the four-dimensional BSE can be reduced to a three-dimensional equation in center of mass frame

$$i\mathcal{M}_{\lambda'_1 \lambda'_2 \lambda_1 \lambda_2}(\mathbf{k}', \mathbf{k}) = i\mathcal{V}_{\lambda'_1 \lambda'_2 \lambda_1 \lambda_2}(\mathbf{k}', \mathbf{k}) + \sum_{\lambda''_1, \lambda''_2} \int \frac{d\mathbf{k}''}{(2\pi)^3} i\mathcal{V}_{\lambda'_1 \lambda'_2 \lambda''_1 \lambda''_2}(\mathbf{k}', \mathbf{k}'') G_0(\mathbf{k}'') i\mathcal{M}_{\lambda''_1 \lambda''_2 \lambda_1 \lambda_2}(\mathbf{k}'', \mathbf{k}),$$

Note: the $i\mathcal{M}$ and $i\mathcal{V}$ are usually real. In the center of mass frame. We choose $\mathbf{k}_2 = \mathbf{k}$ and $\mathbf{k}_1 = -\mathbf{k}$.

In the one-boson-exchange model, the potential can be written as

$$i\mathcal{V} = \sum_{J_e=0} I_e^0 \frac{-\Gamma_{upper} \Gamma_{lower}}{q^2 - m_e^2} f_e(q^2) + \sum_{J_e=1} I_e^1 \frac{\Gamma_{upper}^\mu \Gamma_{lower}^\nu (g^{\mu\nu} - q^\mu q^\nu / m_e^2)}{q^2 - m_e^2} f_e(q^2) \quad (7)$$

where J_e and m_e are the spin and mass of the exchanged meson and only $J_e \leq 1$ are considered. For a discussion on relating the potential kernel to the potential in the Schrödinger equation, see Ref. [He:2014nya].

With the Lagrangians, the interaction vertices can be written straightforwardly by applying the following rules:

- Legs:
 - Scalar meson (spin $S = 0$): 1
 - Vector meson ($S = 1$): ε^μ
 - Baryon ($S = 1/2$): u
 - Baryon ($S = 3/2$): u^μ
- Derivatives: $\partial^\mu \rightarrow -ip^\mu$ (Note: Pay attention to the momentum orientation when applying this rule.)
- γ matrices and $\epsilon^{\mu\nu\rho\lambda}$ tensors remain unchanged.
- An additional factor of i should be included for each vertex.

Partial-wave expansion

To reduce the equation to one-dimensional equation, we apply the partial wave expansion,

$$\begin{aligned} \mathcal{V}_{\lambda'\lambda}(\mathbf{k}', \mathbf{k}) &\equiv \langle \theta' \phi', \lambda' | \mathcal{V} | \theta \phi, \lambda \rangle = \sum_{J'M'JM} \langle \theta' \phi' | J'M', \lambda' \rangle \langle J'M' | \mathcal{V} | JM, \lambda \rangle \langle JM, \lambda | \theta \phi, \lambda \rangle \\ &= \sum_{JM} N_J^2 D_{M\lambda'}^{J*}(\phi', \theta', 0) \mathcal{V}_{\lambda'\lambda}^{JM}(\mathbf{k}', \mathbf{k}) D_{M\lambda}^J(\phi, \theta, 0) = \sum_J N_J^2 D_{\lambda\lambda}^J(\Omega^{-1}\Omega') \mathcal{V}_{\lambda'\lambda}^J(\mathbf{k}', \mathbf{k}). \\ \mathcal{V}_{\lambda'\lambda}^{JM}(\mathbf{k}', \mathbf{k}) &= N_J^2 \int d\Omega' d\Omega D_{M,\lambda'}^J(\phi', \theta', 0) \mathcal{V}_{\lambda'\lambda}(\mathbf{k}', \mathbf{k}) D_{M,\lambda}^{J*}(\phi, \theta, 0). \end{aligned} \quad (8)$$

where $N_J = \sqrt{\frac{2J+1}{4\pi}}$, $\int d\Omega D_{\lambda_1, \lambda_2}^{J*}(\phi, \theta, 0) D_{\lambda'_1, \lambda'_2}^{J'}(\phi, \theta, 0) = N_J^{-2}$, and $\sum_M D_{M\lambda'}^{J*}(\Omega) D_{M\lambda}^J(\Omega') = D_{\lambda\lambda}^J(\Omega^{-1}\Omega')$

To calculate $\mathcal{V}_{\lambda'\lambda}^{JM}(\mathbf{k}', \mathbf{k})$, we adopt a special CMS frame. The momenta are chosen as $\mathbf{k}_2 = (E_2, 0, 0, \mathbf{k})$, $\mathbf{k}_1 = (W - E_2, 0, 0, -\mathbf{k})$ and $\mathbf{k}'_2 = (E'_2, \mathbf{k}' \sin \theta_{k,k'}, 0, \mathbf{k}' \cos \theta_{k,k'})$, $\mathbf{k}'_1 = (W - E_2, -\mathbf{k}' \sin \theta_{k,k'}, 0, -\mathbf{k}' \cos \theta_{k,k'})$ with $\mathbf{k} = |\mathbf{k}|$ and $\mathbf{k}' = |\mathbf{k}'|$.

$$\begin{aligned}
\rightarrow \mathcal{V}_{\lambda'\lambda}(\mathbf{k}', \mathbf{k}) &= \sum_{JM} N_J^2 D_{M\lambda'}^{J*}(0, \theta_{k',k}, 0) \mathcal{V}_{\lambda'\lambda}^{JM}(\mathbf{k}', \mathbf{k}) D_{M\lambda}^J(0, 0, 0). \\
&= \sum_{JM} N_J^2 d_{M\lambda'}^J(\theta_{k',k}) \mathcal{V}_{\lambda'\lambda}^{JM}(\mathbf{k}', \mathbf{k}) \delta_{M\lambda} = \sum_J N_J^2 d_{\lambda,\lambda'}^J(\theta_{k',k}) \mathcal{V}_{\lambda'\lambda}^J(\mathbf{k}', \mathbf{k}) \\
\mathcal{V}_{\lambda'\lambda}^J(\mathbf{k}', \mathbf{k}) &= 2\pi \int d \cos \theta_{k,k'} d_{\lambda,\lambda'}^J(\theta_{k',k}) \mathcal{V}_{\lambda'\lambda}(\mathbf{k}', \mathbf{k}).
\end{aligned} \tag{9}$$

where $\int_{-1}^1 d \cos \theta' d_{\lambda,\lambda'}^{J'}(\theta') d_{\lambda,\lambda'}^J(\theta') = N_J^{-2} / 2\pi \delta_{JJ'}$ are used.

NOTE: Which particle is chosen to parallel to z axis is related to the order of λ and λ' in $d_{\lambda'\lambda}^J(\theta_{k,k'})$, so it can not be chosen arbitrarily. And the definition of helicity is also dependent of the definition of $\mathbf{k}_{1,2}$. Here, $\lambda = \lambda_2 - \lambda_1$ and $\lambda_1 = -s_1$, $\lambda_2 = s_2$. For brevity, however, the subscript λ in $\mathcal{V}$ and $\mathcal{M}$ is used as a shorthand for the pair (λ_1, λ_2) ; this shorthand should be distinguished from the true $\lambda = \lambda_2 - \lambda_1$ appearing in the Wigner d -function.

The scattering amplitudes $\mathcal{M}$ has analogous relations. Now we have the partial wave BS equation,

$$\begin{aligned}
i\mathcal{M}_{\lambda'\lambda}^J(\mathbf{k}', \mathbf{k}) &= N_J^2 \int d\Omega' d\Omega D_{\lambda_R,\lambda'}^{J*}(\phi', \theta', 0) D_{\lambda_R,\lambda}^J(\phi, \theta, 0) \\
&\cdot \left[i\mathcal{V}_{\lambda'\lambda}(\mathbf{k}', \mathbf{k}) + \int \frac{d\mathbf{k}''}{(2\pi)^3} i\mathcal{V}_{\lambda'\lambda''}(\mathbf{k}', \mathbf{k}'') G_0(\mathbf{k}'') i\mathcal{M}_{\lambda''\lambda}(\mathbf{k}'', \mathbf{k}) \right] \\
&= i\mathcal{V}_{\lambda'\lambda}^J(\mathbf{k}', \mathbf{k}) + \int \frac{k''^2 dk''}{(2\pi)^3} i\mathcal{V}_{\lambda'\lambda''}^J(\mathbf{k}', \mathbf{k}'') G_0(\mathbf{k}'') i\mathcal{M}_{\lambda''\lambda}^J(\mathbf{k}'', \mathbf{k})
\end{aligned} \tag{10}$$

Fixed parity

For a helicity state $|J, \lambda\rangle = |J, \lambda_1 \lambda_2\rangle$ fulfill the party property,

$$P|J, \lambda\rangle = P|J, \lambda_1 \lambda_2\rangle = \eta_1 \eta_2 (-1)^{J-s_1-s_2} |J, -\lambda_1 - \lambda_2\rangle \equiv \tilde{\eta} |J - \lambda\rangle \tag{11}$$

The construction of normalized states with parity $\pm$ is now straightforward:

$$\begin{aligned}
|J, \lambda; \pm\rangle &= \frac{1}{\sqrt{2}} (|J, +\lambda\rangle \pm \tilde{\eta} |J, -\lambda\rangle) \\
\Rightarrow P|J, \lambda; \pm\rangle &= \frac{1}{\sqrt{2}} (\tilde{\eta} |J, -\lambda\rangle \pm |J, +\lambda\rangle) = \pm \frac{1}{\sqrt{2}} (\pm \tilde{\eta} |J, -\lambda\rangle + |J, \lambda\rangle) = \pm |J, \lambda; \pm\rangle
\end{aligned} \tag{12}$$

The amplitude with fixed parity is defined as

$$\mathcal{M}_{\lambda'\lambda}^{J\pm} = \langle J, \lambda'; \pm | \mathcal{M} | J, \lambda; \pm \rangle$$

With such definition, we have

$$\mathcal{M}_{\lambda'-\lambda}^{J\pm} = \pm \tilde{\eta} \mathcal{M}_{\lambda'\lambda}^{J\pm} \equiv \eta \mathcal{M}_{\lambda'\lambda}^{J\pm}, \quad \mathcal{M}_{-\lambda'\lambda}^{J\pm} = \pm \tilde{\eta}' \mathcal{M}_{\lambda'\lambda}^{J\pm} \equiv \eta' \mathcal{M}_{\lambda'\lambda}^{J\pm} \tag{13}$$

with $\eta = PP_1P_2(-1)^{J_1+J_2-J}$.

For parity conserving interactions $M = \hat{P}^{-1} M \hat{P}$ follows:

$$\mathcal{M}_{-\lambda'-\lambda}^J = \tilde{\eta} \tilde{\eta}' \mathcal{M}_{\lambda'\lambda}^J \tag{14}$$

We have

$$\mathcal{M}_{\lambda'\lambda}^{J\pm} = \langle J, \lambda'; \pm | \mathcal{M} | J, \lambda; \pm \rangle = \mathcal{M}_{\lambda'\lambda}^J \pm \tilde{\eta} \mathcal{M}_{\lambda'-\lambda}^J = \mathcal{M}_{\lambda'\lambda}^J \pm \tilde{\eta}' \mathcal{M}_{-\lambda'\lambda}^J \tag{15}$$

$$i\mathcal{V}_{\lambda'\lambda}^{JP}(\mathbf{k}', \mathbf{k}) = 2\pi \int d \cos \theta [d_{\lambda\lambda'}^J(\theta) i\mathcal{V}_{\lambda'\lambda}(\mathbf{k}', \mathbf{k}) + \eta d_{-\lambda\lambda'}^J(\theta) i\mathcal{V}_{\lambda'-\lambda}(\mathbf{k}', \mathbf{k})], \tag{16}$$

The potential $\mathcal{V}_{\lambda'\lambda}^{JP}$ has analogous relations.

$$\begin{aligned}
i\mathcal{M}_{\lambda'\lambda}^J &= i\mathcal{V}_{\lambda'\lambda}^J + \sum_{\lambda''} i\mathcal{V}_{\lambda'\lambda''}^J G_0 i\mathcal{M}_{\lambda''\lambda}^J, \quad \eta' i\mathcal{M}_{-\lambda'\lambda}^J = \eta' i\mathcal{V}_{-\lambda'\lambda}^J + \sum_{\lambda''} \eta' i\mathcal{V}_{-\lambda'\lambda''}^J G_0 i\mathcal{M}_{\lambda''\lambda}^J \\
\Rightarrow i\mathcal{M}_{\lambda'\lambda}^{J^P} &= i\mathcal{V}_{\lambda'\lambda}^{J^P} + \sum_{\lambda''} i\mathcal{V}_{\lambda'\lambda''}^{J^P} G_0 i\mathcal{M}_{\lambda''\lambda}^J, \\
&= i\mathcal{V}_{\lambda'\lambda}^{J^P} + \frac{1}{2} \sum_{\lambda''} (i\mathcal{V}_{\lambda'\lambda''}^{J^P} G_0 i\mathcal{M}_{\lambda''\lambda}^J + i\mathcal{V}_{\lambda'-\lambda''}^{J^P} G_0 i\mathcal{M}_{-\lambda''\lambda}^J) \\
&= i\mathcal{V}_{\lambda'\lambda}^{J^P} + \frac{1}{2} \sum_{\lambda''} (i\mathcal{V}_{\lambda'\lambda''}^{J^P} G_0 i\mathcal{M}_{\lambda''\lambda}^J + i\mathcal{V}_{\lambda'\lambda''}^J G_0 \eta'' i\mathcal{M}_{-\lambda''\lambda}^{J^P}) \\
&= i\mathcal{V}_{\lambda'\lambda}^{J^P} + \frac{1}{2} \sum_{\lambda''} i\mathcal{V}_{\lambda'\lambda''}^{J^P} G_0 i\mathcal{M}_{\lambda''\lambda}^{J^P}, \tag{17}
\end{aligned}$$

In fact such relation can be generalized as

$$C_{\lambda'\lambda}^J = \sum_{\lambda''} \mathcal{A}_{\lambda'\lambda''}^J \mathcal{B}_{\lambda''\lambda}^J \Rightarrow C_{\lambda'\lambda}^{J^P} = \frac{1}{2} \sum_{\lambda''} \mathcal{A}_{\lambda'\lambda''}^{J^P} \mathcal{B}_{\lambda''\lambda}^{J^P}, \tag{18}$$

Solution of quasipotential Bethe-Salpeter equation

Independent helicity amplitudes

With Eq. (13), the amplitudes with different helicities are not independent. To reduce the calculation time, we only keep the independent helicity amplitudes as

$$\sum_{\lambda''} \mathcal{A}_{\lambda'\lambda''}^{J^P} \mathcal{B}_{\lambda''\lambda}^{J^P} = \mathcal{A}_{\lambda'0}^{J^P} \mathcal{B}_{0\lambda}^{J^P} + 2 \sum_{\lambda'' \in I_{\neq 0}} \mathcal{A}_{\lambda'\lambda''}^{J^P} \mathcal{B}_{\lambda''\lambda}^{J^P} = 2 \sum_k \mathcal{A}_{\lambda'k}^{J^P} \mathcal{B}_{k\lambda}^{J^P}. \tag{19}$$

where k are the indices for the independent helicity amplitudes, and $I_{\neq 0}$ means the nonzero independent helicities. And we redefine

$$f_{\lambda'} f_{\lambda} \mathcal{A}_{\lambda'\lambda}^{J^P} \rightarrow \mathcal{A}_{ij}^{J^P},$$

with $f_0 = \frac{1}{\sqrt{2}}$, $f_{\lambda \neq 0} = 1$, and with the amplitudes for $\lambda_1 = \lambda_2 = 0$ set to $i = 0$. Here, the subscript ij denotes the amplitudes that include only the independent helicity amplitudes multiplied by the factors f_i and f_j ; these are directly provided by the program. In contrast, the subscript λ (i.e., $\mathcal{A}_{\lambda'\lambda}$) denotes the physical amplitudes, which must be obtained from the program outputs together with the factors f_{λ} .

As shown in Eq. (14), the amplitudes are not independent. If we only keep the independent amplitudes, the equation for definite parity can be written as

$$i\mathcal{M}_{ij}^{J^P} = i\mathcal{V}_{ij}^{J^P} + \sum_k i\mathcal{V}_{ik}^{J^P} G_0 i\mathcal{M}_{kj}^{J^P}. \tag{20}$$

The Bethe-Salpeter equation for partial-wave amplitude with fixed spin-parity J^P reads ,

$$i\mathcal{M}_{ij}^{J^P}(\mathbf{k}', \mathbf{k}) = i\mathcal{V}_{ij}^{J^P}(\mathbf{k}', \mathbf{k}) + \sum_k \int \frac{k''^2 d\mathbf{k}''}{(2\pi)^3} i\mathcal{V}_{ik}^{J^P}(\mathbf{k}', \mathbf{k}'') G_0(k'') i\mathcal{M}_{kj}^{J^P}(\mathbf{k}'', \mathbf{k}). \tag{21}$$

The partial wave potential is defined with the independent helicities as

$$i\mathcal{V}_{ij}^{J^P}(\mathbf{k}', \mathbf{k}'') = f_{\lambda'} f_{\lambda} 2\pi \int d\cos\theta [d_{\lambda\lambda'}^J(\theta) i\mathcal{V}_{\lambda'\lambda}^J(\mathbf{k}', \mathbf{k}) + \eta d_{-\lambda\lambda'}^J(\theta) i\mathcal{V}_{\lambda'-\lambda}^J(\mathbf{k}', \mathbf{k})]. \tag{22}$$

Note here the $f_{\lambda'} f_{\lambda}$ is also incorporated. Additionally, the form factors for the interacting particles are also included in the potential, modifying it as $\mathcal{V} \rightarrow f(k') \mathcal{V} f(k)$. Consequently, the resulting amplitude $\mathcal{M}$ also includes these form factors.

The potential $\mathcal{V}_{\lambda'\lambda}(\mathbf{k}', \mathbf{k})$ is introduced by **fV** function in main file as

```
fV(k, l, SYS, IA0, CHf, CHi, VVertex)
```

where **k** and **l** are for the momenta and helicities of final and initial particles. **SYS** is for the system information. **IA0** is for the interaction information., **CHf** and **CHi** are for the information of final and initial channels. **VVertex** is a function that returns the explicit form of the interaction or vertices, as defined in **main.jl**.

Transition of $\mathcal{V}_{\lambda'\lambda}(\mathbf{k}', \mathbf{k})$ to $\mathcal{V}_{ij}^{JP}(\mathbf{k}', \mathbf{k}'')$ performed in **qBSE.fKernel** which is an internal function.

Treatment of the singularity

Now We have a integral equation with singularity in $G_0 = \frac{1}{2E_2[(W-E_2)^2-E_1^2]} = \frac{1}{2E_2[(W-E_2-E_1+i\epsilon)(W-E_2+E_1)]}$ at $W = E_1 + E_2$. This singularity can be isolated as,

$$i\mathcal{M}^{JP}(\mathbf{k}', \mathbf{k}) = i\mathcal{V}^{JP}(\mathbf{k}, \mathbf{k}') + \int \frac{\mathbf{k}''^2 d\mathbf{k}''}{(2\pi)^3} i\mathcal{V}^{JP}(\mathbf{k}, \mathbf{k}'') G_0(\mathbf{k}'') i\mathcal{M}^{JP}(\mathbf{k}'', \mathbf{k}') \quad (23)$$

Using $\frac{1}{x \pm i\epsilon} = \mathcal{P} \frac{1}{x} \mp i\pi\delta(x)$, if $W > m_1 + m_2$ and $\bar{q} < q_{max}$ ($\bar{q} = \frac{1}{2W} \sqrt{[W^2 - (m_1 + m_2)^2][W^2 - (m_1 - m_2)^2]}$ is onshell momentum),

$$\int_0^{q_{max}} dq F(q) \frac{1}{W-E_1-E_2+i\epsilon} = \mathcal{P} \int_0^{q_{max}} dq F(q) \frac{1}{W-E_1-E_2} - i\pi\rho(\bar{q})$$

$$\rho(\bar{q}) = F(\bar{q})\delta(W-E_1-E_2) = F(\bar{q}) \frac{\delta(q-\bar{q})}{|(-\frac{1}{2})(\frac{2\bar{q}}{E_1} + \frac{2\bar{q}}{E_2})|} = F(\bar{q}) \frac{\bar{E}_1 \bar{E}_2}{\bar{q}W} \delta(q-\bar{q})$$

$$\text{Using } \frac{\bar{q}^2 - q^2}{W-E_1-E_2} \Big|_{q \rightarrow \bar{q}} = \frac{-2q}{-\frac{q}{E_1} - \frac{q}{E_2}} \Big|_{q \rightarrow \bar{q}} = \frac{2\bar{E}_1 \bar{E}_2}{W},$$

$$\begin{aligned} \mathcal{P} \int_0^{q_{max}} dq F(q) \frac{1}{W-E_1-E_2} &= \int_0^{q_{max}} dq \left[F(q) \frac{\bar{q}^2 - q^2}{W-E_1-E_2} - F(\bar{q}) \frac{\bar{q}^2 - q^2}{W-E_1-E_2} \Big|_{q \rightarrow \bar{q}} \right] \frac{1}{\bar{q}^2 - q^2} + \\ \mathcal{P} \int_0^{q_{max}} dq F(\bar{q}) \frac{\bar{q}^2 - q^2}{W-E_1-E_2} \Big|_{q \rightarrow \bar{q}} \frac{1}{\bar{q}^2 - q^2} &= \int_0^{q_{max}} dq \left[F(q) \frac{1}{W-E_1-E_2} - F(\bar{q}) \frac{2\bar{E}_1 \bar{E}_2}{W} \frac{1}{\bar{q}^2 - q^2} \right] + \\ F(\bar{q}) \frac{2\bar{E}_1 \bar{E}_2}{W} \frac{1}{2\bar{q}} \ln \left| \frac{q_{max} + \bar{q}}{q_{max} - \bar{q}} \right| \end{aligned}$$

Hence, we have

$$\begin{aligned} \int_0^{q_{max}} dq F(q) \frac{1}{W-E_1-E_2+i\epsilon} &= \int_0^{q_{max}} dq \left[F(q) \frac{1}{W-E_1-E_2} - F(\bar{q}) \frac{2\bar{E}_1 \bar{E}_2}{W} \frac{1}{\bar{q}^2 - q^2} \right] + \\ F(\bar{q}) \frac{\bar{E}_1 \bar{E}_2}{\bar{q}W} \ln \left| \frac{q_{max} + \bar{q}}{q_{max} - \bar{q}} \right| - i\pi F(\bar{q}) \frac{\bar{E}_1 \bar{E}_2}{\bar{q}W} \\ &= \int_0^{q_{max}} dq F(q) \frac{1}{W-E_1-E_2} + F(\bar{q}) \frac{\bar{E}_1 \bar{E}_2}{\bar{q}W} \left[-2\bar{q} \int_0^{q_{max}} dq \frac{1}{\bar{q}^2 - q^2} + \ln \left| \frac{q_{max} + \bar{q}}{q_{max} - \bar{q}} \right| - i\pi \right] \end{aligned}$$

$$\text{If } q_{max} \rightarrow \infty, \ln \left| \frac{q_{max} + q_0}{q_{max} - q_0} \right| \rightarrow 0.$$

Now

$$F(\mathbf{k}'') = \frac{\mathbf{k}''^2}{(2\pi)^3} i\mathcal{V}^{JP}(\mathbf{k}, \mathbf{k}'') \frac{1}{2E_2[W-E_2+E_1]} i\mathcal{M}^{JP}(\mathbf{k}'', \mathbf{k}') \rightarrow F(\bar{\mathbf{k}}'') = \frac{\bar{\mathbf{k}}''^2}{(2\pi)^3} i\mathcal{V}_o^{JP}(\mathbf{k}, \bar{\mathbf{k}}'') \frac{1}{4E_2 E_1} i\mathcal{M}_o^{JP}(\bar{\mathbf{k}}'', \mathbf{k}')$$

$$\begin{aligned} &\int \frac{\mathbf{k}''^2 d\mathbf{k}''}{(2\pi)^3} i\mathcal{V}^{JP}(\mathbf{k}, \mathbf{k}'') G_0(\mathbf{k}'') i\mathcal{M}^{JP}(\mathbf{k}'', \mathbf{k}') \\ &= \int_0^{\mathbf{k}''_{max}} \frac{\mathbf{k}''^2 d\mathbf{k}''}{(2\pi)^3} i\mathcal{V}^{JP} G_0 i\mathcal{M}^{JP} + \frac{\bar{\mathbf{k}}''}{32\pi^3 W} i\mathcal{V}_o^{JP}(\mathbf{k}, \bar{\mathbf{k}}'') i\mathcal{M}_o^{JP}(\bar{\mathbf{k}}'', \mathbf{k}') \left[2\bar{\mathbf{k}}'' \int_0^{\mathbf{k}''_{max}} d\mathbf{k}'' \frac{1}{\mathbf{k}''^2 - \bar{\mathbf{k}}''^2} (24) \ln \left(\frac{\mathbf{k}''_{max} + \bar{\mathbf{k}}''}{\mathbf{k}''_{max} - \bar{\mathbf{k}}''} \right) \right] \end{aligned}$$

We have

$$Im\ G = -\rho/2 = -\frac{k''}{32\pi^2 W}. \quad (25)$$

It should be noted that in the region $W < m_1 + m_2$, the potential remains real, since no imaginary component emerges in this energy range.

It suggests the unitary is satisfied if the potential $i\mathcal{V}$ is real.

$$-T^\dagger \rho T = 2T^\dagger ImG T = 2T^\dagger (-ImT^{-1})T = 2T^\dagger \frac{1}{2i}(T^{\dagger-1} - T^{-1})T = i(T - T^\dagger) \quad (26)$$

where $T = i\mathcal{M}$.

Transformation to a matrix equation

With the Gauss discretization, the one-dimensional equation can be transformed as a matrix equation as

$$i\mathcal{M}_{ik}^{J^P} = i\mathcal{V}_{ik}^{J^P} + \sum_{j=0}^N i\mathcal{V}_{ij}^{J^P} G_j i\mathcal{M}_{jk}^{J^P} \Rightarrow M^{J^P} = V^{J^P} + V^{J^P} G M^{J^P} \quad (27)$$

$$G_j = \begin{cases} \frac{\bar{q}}{32\pi^3 W} \left[2\bar{q} \sum_j \frac{w(q_j)}{q_j^2 - \bar{q}^2} + \ln \left| \frac{k''_{max} + \bar{k}''}{k''_{max} - \bar{k}''} \right| - i\pi \right] & \text{for } j = 0, \text{ if } Re(W) > m_1 + m_2, \\ \frac{w(q_j)}{(2\pi)^3} \frac{q_j^2}{2E(q_j)[(W - E(q_j))^2 - \omega^2(q_j)]} & \text{for } j \neq 0 \end{cases}$$

If $q_{max} \rightarrow \infty$

$$G_j = \begin{cases} -\frac{i\bar{q}}{32\pi^2 W} + \sum_j \left[\frac{w(q_j)}{(2\pi)^3} \frac{\bar{q}^2}{2W(q_j^2 - \bar{q}^2)} \right] & \text{for } j = 0, \text{ if } Re(W) > m_1 + m_2, \\ \frac{w(q_j)}{(2\pi)^3} \frac{q_j^2}{2E(q_j)[(W - E(q_j))^2 - \omega^2(q_j)]} & \text{for } j \neq 0 \end{cases}$$

where $\bar{q} = \frac{1}{2W} \sqrt{[W^2 - (m_1 + m_2)^2][W^2 - (m_1 - m_2)^2]}$. The indices i, j, k is for discrete momentum values, independent helicities, and coupled channels.

The propagator is calculated in `qBSE.fProp` which is an internal function.

The default dimension is $[G] = 1$. Recalling that a factor of $2m$ should be included if a constituent particle is a fermion, we have $[G] = \text{GeV}^{n_f} \rightarrow [V] = [M] = \text{GeV}^{-n_f}$, with n_f being the number of fermions. Therefore, under our convention where $\bar{u}u = 1$, the dimension of the potential must satisfy the above requirements. This criterion can be employed to verify the consistency of the Lagrangian and the derived potential.

Hence, for the channels above its thresholds, the matrix have an extra dimension. We take two channel as example to explain the coupled-channel equation. The region of W is divided as $W < m_1$, $m_1 < W < m_2$ and $W > m_2$.

$$V = \begin{pmatrix} V_{11}^{NN} & V_{12}^{NN} \\ V_{21}^{NN} & V_{22}^{NN} \end{pmatrix}, \quad G = \begin{pmatrix} G_1^N & 0 \\ 0 & G_2^N \end{pmatrix}, \quad W < m_1, \quad (28)$$

$$V = \begin{pmatrix} V_{11}^{N+1N+1} & V_{12}^{N+1N} \\ V_{21}^{NN+1} & V_{22}^{NN} \end{pmatrix}, \quad G = \begin{pmatrix} G_1^{N+1} & 0 \\ 0 & G_2^N \end{pmatrix}, \quad m_1 < W < m_2 \quad (29)$$

$$V = \begin{pmatrix} V_{11}^{N+1N+1} & V_{12}^{N+1N+1} \\ V_{21}^{N+1N+1} & V_{22}^{N+1N+1} \end{pmatrix}, \quad G = \begin{pmatrix} G_1^{N+1} & 0 \\ 0 & G_2^{N+1} \end{pmatrix}, \quad W > m_2 \quad (30)$$

The informations about the dimensions are calculated in `qBSE.WORKSPACE`. The matrix V and G are calculated in `qBSE.srAB`. Note that such function is internal function and not used by the user.

For old code

Attention: The following details are specific to the old version of the code, which includes Fortran code and Julia code versions prior to v0.2.4. In the new version, the treatment described below is obsolete.

In old code, we choose $\hat{V}^{J^P} = V^{J^P}/4\pi$, $\hat{G} = 4\pi G$, and $\hat{M}^{J^P} = M^{J^P}/4\pi$. The form factors are also included in to the amplitudes and the potential kernel. Hence, the qBSE becomes

$$\hat{M}^{J^P} = \hat{V}^{J^P} + \hat{V}^{J^P} G \hat{M}^{J^P}. \quad (31)$$

Such convention is consistent with that in the chiral unitary approach.

$$\begin{aligned} \hat{V}^{J^P} &= V^{J^P}/4\pi = i\mathcal{V}_{\lambda'\lambda''}^{J^P}(\mathbf{p}', \mathbf{p}'')/4\pi = f_{\lambda'} f_{\lambda} \mathcal{V}_{\lambda'\lambda}^{J^P}(\mathbf{p}', \mathbf{p})/4\pi \\ &= \frac{1}{2} f_{\lambda'} f_{\lambda} \int d\cos\theta [d_{\lambda\lambda'}^J(\theta) i\mathcal{V}_{\lambda'\lambda}(\mathbf{p}', \mathbf{p}) + \eta d_{-\lambda\lambda'}^J(\theta) i\mathcal{V}_{\lambda'-\lambda}(\mathbf{p}', \mathbf{p})], \\ \hat{G}_j &= \begin{cases} -\frac{i\bar{q}}{8\pi W} + \sum_j \left[\frac{w(q_j)}{2\pi^2} \frac{\bar{q}^2}{2W(q_j^2 - \bar{q}^2)} \right] & \text{for } j = 0, \text{ if } \text{Re}(W) > m_1 + m_2, \\ \frac{w(q_j)}{2\pi^2} \frac{q_j^2}{2E(q_j)[(W - E(q_j))^2 - \omega^2(q_j)]} & \text{for } j \neq 0 \end{cases} \end{aligned} \quad (32)$$

Pole search

To find a bound state or resonance, the singularities should be searched at the pole of the $M(z) = 0$ in the complex plane after analytic continuation total energy W into the complex plane as z .

Since $E = \sqrt{m_1^2 + p^2} + \sqrt{m_2^2 + p^2}$, the p -plane correspond to two Riemann sheets for E . The bound state is located in the first Riemann sheet while the resonances located in the second Riemann sheet.

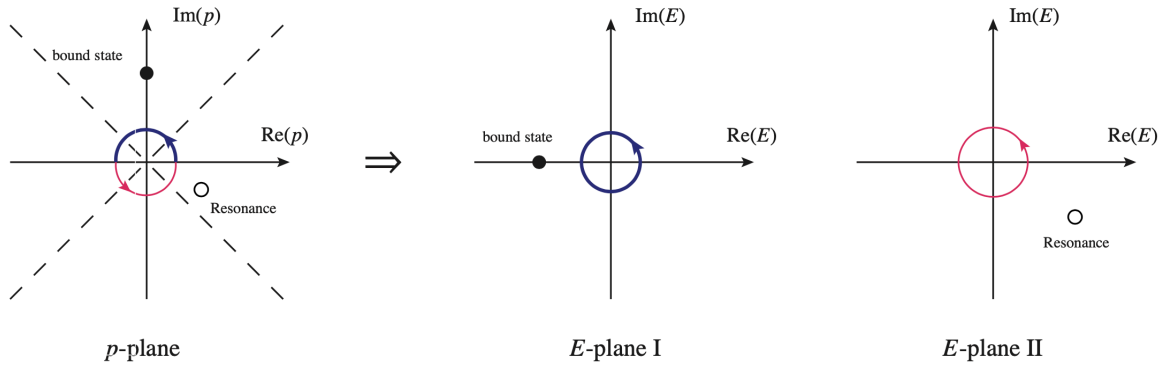


Figure F.1: Singularity classification on the p -plane and E -plane physical (I) and unphysical (II) Riemann sheets in the one channel case.

From the above figure, the resonances should be found with $\text{Im}(q) < 0$. The potential V is dependent on the E .

After extend the energy in the center of mass frame W into complex energy plane as z , the pole can be found by variation of z to satisfy

$$|1 - V(z)G(z)| = 0 \quad (33)$$

with $z = E_R + i\Gamma_R/2$.

The $|1 - V(z)G(z)|$ in complex energy plane is calculated in `qBSE.res` and the pole can be found with function `qBSE.showPoleInfo`.

As shown above, the propagator acquires an imaginary part in the region $W > m_1 + m_2$, specifically $-i\rho/2$. According to the Schwarz reflection principle, if a function $f(z)$ is analytic in a region of the complex plane that includes a portion of the real axis where f is real, then it satisfies $[f(z^*)]^* = f(z)$. Since the propagator G satisfies these conditions, we have, for $\text{Re}(z) > m_1 + m_2$,

$$G(z - i\epsilon) = G^*(z + i\epsilon) = G(z + i\epsilon) + i\rho.$$

The value of the propagator at the lower edge of the branch cut on the first Riemann sheet, $G^I(z - i\epsilon)$, coincides with the upper edge of the second Riemann sheet, $G^{II}(z + i\epsilon)$. Therefore,

$$G^{II}(z + i\epsilon) = G^I(z - i\epsilon) = G^I(z + i\epsilon) + i\rho.$$

Before analytically continuing into the complex plane, we adopt the convention that on the first Riemann sheet, the propagator is real for $W < m_1 + m_2$, and has an imaginary part $-i\rho/2$ for $W > m_1 + m_2$. The second Riemann sheet is defined as the region where the propagator has an imaginary part of $i\rho/2$, i.e., shifted by $i\rho$ relative to the first sheet.

When searching for the poles, we use the first Riemann sheet for $W < m_1 + m_2$, and the second Riemann sheet for $W > m_1 + m_2$. In the code, the `lRm` in `qn` is chosen as `0` in this case. If only first or second Riemann sheet is considered, `lRm` is set to `1` or `2`.

In the code, the `lRm` can be set to a `Int64` number `0`, `1`, or `2` for all channels, or `(n1, n2, n2, ...)` for each channel, where `n1`, `n2`, etc. are the Riemann sheet numbers for each channel.

Attention: Physical observables are computed on the first Riemann sheet—more precisely, along the real axis (`lRm=1`). Therefore, for $W < m_1 + m_2$, the treatment is consistent with that used in pole searches. However, for $W > m_1 + m_2$, the pole search is carried out on the second Riemann sheet, where the propagator has an imaginary part of $i\rho/2$, in contrast to $-i\rho/2$ on the first sheet. This leads to a relative phase difference between the two Riemann sheets along the real axis.

2-2 cross section

With the obtained amplitude M^{JP} , we can also calculate the physical observable. Note that all physical observable are at real axis, we choose the onshell momentum as

$$M_{ij}(z) = \{[(1 - VG)^{-1}]V\}_{ij} \quad (34)$$

with i and k chosen as the onshell momentum, that is, 0 dimension for G , and extra dimension for V .

The $|M|^2$ for each channel is calculated in `qBSE.res`.

The cross section for the channel considered

The cross section, denoted by $d\sigma$, can be expressed in terms of amplitudes, $\mathcal{M}$, as follows:

$$d\sigma = F \frac{1}{S} \frac{1}{\tilde{j}_1 \tilde{j}_2} \sum |\mathcal{M}|^2 d\Phi = (2\pi)^{4-3n} F \frac{1}{S} \frac{1}{\tilde{j}_1 \tilde{j}_2} \sum |\mathcal{M}|^2 dR$$

The flux factor F for the cross section is given by:

$$F = \frac{1}{2E_1 2E_2 v_{12}} = \frac{1}{4[(p_1 \cdot p_2)^2 - m_1^2 m_2^2]^{1/2}} \frac{|p_1 \cdot p_2|}{p_1^0 p_2^0}$$

In the laboratory or center of mass frame, the relation $\vec{p}_1^2 \vec{p}_2^2 = (\vec{p}_1 \cdot \vec{p}_2)^2$ is utilized. In the laboratory frame, the term $\frac{|p_1 \cdot p_2|}{p_1^0 p_2^0}$ simplifies to 1. In center of mass frame, $v_{12} = \frac{k(E_1 + E_2)}{E_1 E_2} = \frac{k\sqrt{s}}{E_1 E_2}$. Additionally, if a boson or zero-mass spinor particle is replaced with a non-zero mass spinor particle, the factor $1/2$ is replaced with the mass of the particle, m , due to convention $\bar{u}u = 1$ adopted. The total symmetry factor S is given by $\prod_i m_i!$ if there are m_i identical particles.

For the open channel, the cross section can be obtained as

$$\frac{d\sigma}{d\Omega} = \frac{1}{\tilde{j}_1 \tilde{j}_2} \frac{1}{64\pi^2 s} \frac{k'}{k} \sum_{\lambda'\lambda} |i\mathcal{M}_{\lambda'\lambda}(\mathbf{k}', \mathbf{k})|^2. \quad (35)$$

where j_1 and j_2 is the spin of the initial particles, and we define $\tilde{j} = 2j + 1$.

The total cross section can be written as

$$\begin{aligned}\sigma &= \frac{1}{\tilde{j}_1 \tilde{j}_2} \frac{1}{64\pi^2 s} \frac{k'}{k} \sum_{J, \lambda' \lambda} N_J^2 |i\mathcal{M}_{\lambda' \lambda}^J(\mathbf{k}', \mathbf{k})|^2 = \frac{1}{\tilde{j}_1 \tilde{j}_2} \frac{1}{64\pi^2 s} \frac{k'}{k} \sum_{J, \lambda' \lambda} N_J^2 \left| \frac{1}{2} i\mathcal{M}_{\lambda' \lambda}^{J^P}(\mathbf{k}', \mathbf{k}) \right|^2 \\ &= \frac{1}{\tilde{j}_1 \tilde{j}_2} \frac{1}{64\pi^2 s} \frac{k'}{k} \sum_{J^P, ij} N_J^2 \left| M_{ij}^{J^P} \right|^2.\end{aligned}\quad (36)$$

Here, $\mathbf{k}'$ and $\mathbf{k}$ are onshell momenta, so we only choose ij for the onshell momenta. Since we adopt $\mathcal{V} \rightarrow f(\mathbf{k}') \mathcal{V} f(\mathbf{k})$, the amplitudes $M^{J^P} \rightarrow f(\mathbf{k}') M^{J^P} f(\mathbf{k})$. The form factors vanish due to onshellness for initial and final states of a scattering.

$$M_{\lambda' \lambda}^{J^\pm} = M_{\lambda' \lambda}^J \pm \tilde{\eta}' M_{-\lambda' \lambda}^J = M_{\lambda' \lambda}^J \pm \tilde{\eta} M_{\lambda' -\lambda}^J$$

$$M_{\lambda' \lambda}^J = \frac{1}{2} (M_{\lambda' \lambda}^{J^+} + M_{\lambda' \lambda}^{J^-}), M_{\lambda' -\lambda}^J = \frac{1}{2\tilde{\eta}} (M_{\lambda' \lambda}^{J^+} - M_{\lambda' \lambda}^{J^-})$$

$$M_{\lambda' 0}^J = \frac{1}{2} (\delta_{\tilde{\eta}' 1} M_{\lambda' 0}^{J^+} + \delta_{\tilde{\eta}' -1} M_{\lambda' 0}^{J^-})$$

$$|M_{\lambda' 0}^J|^2 = \left| \frac{1}{2} (\delta_{\tilde{\eta}' 1} M_{\lambda' 0}^{J^+} + \delta_{\tilde{\eta}' -1} M_{\lambda' 0}^{J^-}) \right|^2 = \delta_{\tilde{\eta}' 1} \left| \frac{1}{2} M_{\lambda' 0}^{J^+} \right|^2 + \delta_{\tilde{\eta}' -1} \left| \frac{1}{2} M_{\lambda' 0}^{J^-} \right|^2 = \sum_P \left| \frac{1}{2} M_{\lambda' 0}^{J^P} \right|^2$$

$$\begin{aligned}\sigma &\propto \sum_{J, \lambda' \lambda} |M_{\lambda' \lambda}^J|^2 = \sum_{J, \lambda'} |M_{\lambda' 0}^J|^2 + \sum_{J, \lambda', \lambda \in I_{\neq 0}} [|M_{\lambda' \lambda}^J|^2 + |M_{\lambda' -\lambda}^J|^2] \\ &= \sum_{J^P, \lambda'} \left| \frac{1}{2} M_{\lambda' 0}^{J^P} \right|^2 + \sum_{J, \lambda', \lambda \in I_{\neq 0}} \left[\frac{1}{4} |M_{\lambda' \lambda}^{J^+} + M_{\lambda' \lambda}^{J^-}|^2 + \frac{1}{4} |M_{\lambda' \lambda}^{J^+} - M_{\lambda' \lambda}^{J^-}|^2 \right] \\ &= \sum_{J^P, \lambda'} \left| \frac{1}{2} M_{\lambda' 0}^{J^P} \right|^2 + \frac{1}{2} \sum_{J^P, \lambda', \lambda \in I_{\neq 0}} |M_{\lambda' \lambda}^{J^P}|^2 = \sum_{J^P, \lambda' \lambda} \left| \frac{1}{2} M_{\lambda' \lambda}^{J^P} \right|^2 \\ &= \sum_{J^P} \left| \frac{1}{2} M_{00}^{J^P} \right|^2 + \sum_{J^P, \lambda' \in I_{\neq 0}} 2 \left| \frac{1}{2} M_{\lambda' 0}^{J^P} \right|^2 + \sum_{J^P, \lambda', \lambda \in I_{\neq 0}} \frac{1}{2} |M_{\lambda' \lambda}^{J^P}|^2 \\ &= \sum_{J^P} \left| \frac{1}{2} M_{00}^{J^P} \right|^2 + \sum_{J^P, \lambda' \in I_{\neq 0}} \left| \frac{1}{\sqrt{2}} M_{\lambda' 0}^{J^P} \right|^2 + \sum_{J^P, \lambda \in I_{\neq 0}} \left| \frac{1}{\sqrt{2}} M_{0\lambda}^{J^P} \right|^2 + \sum_{J^P, \lambda' \in I_{\neq 0} \lambda \in I_{\neq 0}} |M_{\lambda' \lambda}^{J^P}|^2 \\ &= \sum_{J^P} \left| \frac{1}{2} M_{00}^{J^P} \right|^2 + \sum_{J^P, \lambda' \in I_{\neq 0}} \left| \frac{1}{\sqrt{2}} M_{\lambda' 0}^{J^P} \right|^2 + \sum_{J^P, \lambda \in I_{\neq 0}} \left| \frac{1}{\sqrt{2}} M_{0\lambda}^{J^P} \right|^2 + \sum_{J^P, \lambda' \in I_{\neq 0} \lambda \in I_{\neq 0}} |M_{\lambda' \lambda}^{J^P}|^2 \\ &= \sum_{J^P, \lambda' \in I \lambda \in I} |f_{\lambda'} f_{\lambda} M_{\lambda' \lambda}^{J^P}|^2 \rightarrow \sum_{J^P, ij} |M_{ij}^{J^P}|^2\end{aligned}\quad (37)$$

The cross section for certain channel is calculated with `qBSE.simpleXsection`

Argand plot

The amplitudes can be written as

$$i\mathcal{M}(\mathbf{k}', \mathbf{k}) = -8\pi\sqrt{s} f(\mathbf{k}', \mathbf{k}) = -\frac{8\pi\sqrt{s}}{|\mathbf{k}|} \sum_{JM} N_J^2 D_{\lambda_R, \lambda}^{J*}(\phi', \theta', 0) a_{\lambda \lambda'}^J(|\mathbf{k}'|, |\mathbf{k}|) D_{M, \lambda'}^J(\phi, \theta, 0) \quad (38)$$

where $a^J = -\frac{|\mathbf{k}|}{8\pi\sqrt{s}} \mathcal{M}^J(|\mathbf{k}|)$, which can be displayed as a trajectory in an Argand plot.

Three body decay

kinematics

Lorentz boost

Here, we consider an process $Y \rightarrow X m_3 \dots m_n \rightarrow [m_1 m_2] m_3 \dots m_n$. To study a $1 \rightarrow n$ decay with the qBSE, we need consider the center of mass frame (CMS) of Y (which is also the laboratory frame in this issue) and the $m_1 m_2$ where the qBSE is applied. The momenta of initial and final particles in the CMS of Y , remarked as lab , are

$$P^{lab} = (W, 0, 0, 0), \quad p_i^{lab} = (E_i^{lab}, \mathbf{p}_i^{lab}).$$

The Lorentz boost from $(m, \mathbf{0})$ to $(E, \mathbf{k})$,

$$\Lambda_\nu^\mu = \frac{1}{m} \begin{pmatrix} E(\mathbf{k}) & k_x & k_y & k_z \\ k_x & m + \frac{k_x k_x}{E+m} & \frac{k_x k_y}{E+m} & \frac{k_x k_z}{E+m} \\ k_y & \frac{k_y k_x}{E+m} & m + \frac{k_y k_y}{E+m} & \frac{k_y k_z}{E+m} \\ k_z & \frac{k_z k_x}{E+m} & \frac{k_z k_y}{E+m} & m + \frac{k_z k_z}{E+m} \end{pmatrix}.$$

With the Lorentz boost the momenta for particle 23 in the laboratory frame $(E_{23}^{lab}, \mathbf{p}_{23}^{lab})$ can be written with the momenta in the CMS of particles 23 $(M_{23}, \mathbf{0})$ as $p^{lab} = \Lambda(E_{23}^{lab}, \mathbf{p}_{23}^{lab}) p^{cm}$,

$$\begin{aligned} \mathbf{p}^{lab} &= \mathbf{p}^{cm} + \frac{\mathbf{p}_{23}^{lab}}{M_{23}} \left[\frac{\mathbf{p}_{23}^{lab} \cdot \mathbf{p}^{cm}}{E_{23}^{lab} + M_{23}} + p^{0cm} \right], \\ p^{0lab} &= \frac{1}{M_{23}} [E_{23}^{lab} p^{0cm} + \mathbf{p}_{23}^{lab} \cdot \mathbf{p}^{cm}], \end{aligned} \quad (39)$$

where $M_{23} = \sqrt{(p_2^{lab} + p_3^{lab})^2} = \sqrt{(p_2^{cm} + p_3^{cm})^2}$, $E_{23}^{lab} = E_2^{lab} + E_3^{lab}$ (onshell) and $E_{23}^{lab} = W - E_3^{lab} - \dots - E_n^{lab}$ (offshell).

The momenta in CMS of 23 can also be written with the momentum in laboratory frame as $p = \Lambda(E_{23}^{lab}, -\mathbf{p}_{23}^{lab}) p^{lab}$

$$\begin{aligned} \mathbf{p}^{cm} &= \mathbf{p}^{lab} - \frac{\mathbf{p}_{23}^{lab}}{M_{23}} \left[-\frac{\mathbf{p}_{23}^{lab} \cdot \mathbf{p}^{lab}}{E_{23}^{lab} + M_{23}} + p^{0lab} \right], \\ p^{0cm} &= \frac{1}{M_{23}} [E_{23}^{lab} p^{0lab} - \mathbf{p}_{23}^{lab} \cdot \mathbf{p}^{lab}]. \end{aligned} \quad (40)$$

The Lorentz boost is performed by `Xs.LorentzBoost`.

Amplitude

Because the $|\mathcal{M}|^2$ is invariant in different reference frame, the amplitude for the direct decay can be written with the momenta in cm frame of partilces 1 and 2 obtained with Lorentz boost, as (here, we ignore the notation cm)

$$\begin{aligned}
i\mathcal{M}_{\lambda'_1, \lambda'_2, \lambda'_3; \lambda}^d(p'_1, p'_2, p'_3) &= i\mathcal{A}_{\lambda'_1, \lambda'_2, \lambda'_3; \lambda}(p'_1, p'_2, p'_3) = i\mathcal{A}_{\lambda'_1, \lambda'_2, \lambda'_3; \lambda}(\Omega'_2, \Omega'_3, M_{12}), \\
&= \sum_{JM} N_J D_{M\lambda'_{21}}^{J*}(\Omega'_2) i\mathcal{A}_{\lambda'_1, \lambda'_2, \lambda'_3; \lambda}^{JM}(\Omega'_3, M_{12}), \quad \text{for onshell} \\
i\mathcal{M}_{\lambda''_1 \lambda''_2; \lambda'_3; \lambda}^d(p''_1, p''_2, p'_3) &= i\mathcal{A}_{\lambda'_1 \lambda'_2; \lambda'_3; \lambda}(\Omega''_2, p''_2, \Omega'_3, M_{12}) \\
&= \sum_{JM} N_J D_{M\lambda'_{21}}^{J*}(\Omega''_2) i\mathcal{A}_{\lambda''_1 \lambda''_2; \lambda'_3; \lambda}^{JM}(p''_2, \Omega_3, M_{12}), \quad \text{for offshell} \\
i\mathcal{A}_{\lambda''_1 \lambda''_2; \lambda'_3; \lambda}^{JM}(\Omega'_3, M_{12}, p''_2) &= N_J \int d\Omega''_2 D_{M, \lambda'_{21}}^J(\Omega''_2) i\mathcal{A}_{\lambda''_1, \lambda''_2; \lambda'_3; \lambda}(\Omega''_2, p''_2, \Omega'_3, M_{12}) \\
i\mathcal{M}_{\lambda'_1, \lambda'_2; \lambda'_3; \lambda}^Z(p'_1, p'_2, p'_3) &= i \int \frac{d^4 p''_2}{(2\pi)^4} \mathcal{T}_{\lambda'_1, \lambda'_2}(p'_1, p'_2; p''_1, p''_2) G(p''_2) \mathcal{A}_{\lambda'_3; \lambda}(p''_1, p''_2, p'_3) \\
&= \sum_{\lambda''_1 \lambda''_2} \int \frac{d^3 p''_2}{(2\pi)^3} i\mathcal{T}_{\lambda'_1, \lambda'_2; \lambda''_1, \lambda''_2}(p'_1, p'_2; p''_1, p''_2) G_0(p''_2) i\mathcal{A}_{\lambda''_1, \lambda''_2; \lambda'_3; \lambda}(p''_1, p''_2, p'_3) \\
&= \sum_{\lambda''_1 \lambda''_2} \int \frac{d^3 p''_2}{(2\pi)^3} i\mathcal{T}_{\lambda'_1, \lambda'_2; \lambda''_1, \lambda''_2}(\Omega'_2, \Omega''_2, p''_2, M_{12}) G_0(p''_2) i\mathcal{A}_{\lambda''_1, \lambda''_2; \lambda'_3; \lambda}(\Omega''_2, p''_2, \Omega'_3, M_{12}) \\
&= \sum_{\lambda''_1 \lambda''_2} \int \frac{p''_2{}^2 dp''_2 d\Omega''_2}{(2\pi)^3} \sum_{J'M'} N_{J'}^2 D_{M'\lambda'_{21}}^{J'*}(\Omega'_2) i\mathcal{T}_{\lambda'_1, \lambda'_2; \lambda''_1, \lambda''_2}^{J'M'}(p''_2, M_{12}) D_{M'\lambda'_{21}}^{J'}(\Omega''_2) \\
&\quad \cdot G_0(p''_2) \sum_{JM} N_J D_{M\lambda'_{21}}^{J*}(\Omega''_2) i\mathcal{A}_{\lambda''_1, \lambda''_2; \lambda'_3; \lambda}^{JM}(p''_2, \Omega'_3, M_{12}) \\
&= \sum_{\lambda''_1 \lambda''_2} \int \frac{p''_2{}^2 dp''_2 d\Omega''_2}{(2\pi)^3} \sum_{JM} N_J D_{M\lambda'_{21}}^{J*}(\Omega'_2) i\mathcal{T}_{\lambda'_1, \lambda'_2; \lambda''_1, \lambda''_2}^J(p''_2, M_{12}) G_0(p''_2) i\mathcal{A}_{\lambda''_1, \lambda''_2; \lambda'_3; \lambda}^{JM}(p''_2, \Omega'_3, M_{12}) \\
&= \sum_{JM} N_J D_{M\lambda'_{21}}^{J*}(\Omega'_2) \sum_{\lambda''_1 \lambda''_2} \int \frac{p''_2{}^2 dp''_2}{(2\pi)^3} i\mathcal{T}_{\lambda'_1, \lambda'_2; \lambda''_1, \lambda''_2}^J(p''_2, M_{12}) G_0(p''_2) i\mathcal{A}_{\lambda''_1, \lambda''_2; \lambda'_3; \lambda}^{JM}(p''_2, \Omega'_3, M_{12}) \\
&\equiv i\mathcal{M}_{\lambda'_1, \lambda'_2; \lambda'_3; \lambda}^Z(\Omega'_2, \Omega'_3, M_{12}). \tag{41}
\end{aligned}$$

Here, the partice 3 can be extended to $3 \dots n$.

Note: when consider the rescattering of different particles, the different cm frames should be adopted.

Decay width

The phase space is given by

$$d\Phi = (2\pi)^4 \delta^4(P - \sum_{i=1}^n p_i) \prod_{i=1}^n \frac{d^3 p_i}{2E_i (2\pi)^3} \tag{42}$$

We consider Monte-Carlo method to generate the event.

$$d\Gamma = \frac{1}{2E} \sum |\mathcal{M}|^2 d\Phi = \frac{1}{2E} \sum |\mathcal{M}|^2 (2\pi)^{4-3n} dR \tag{43}$$

The distribution can be calculated with [qBSE.Xsection](#).

Here we consider a process with ij rescattering (k denote other particles).

$$\begin{aligned}
i\mathcal{M}_{\lambda'_i, \lambda'_j; \lambda'_k; \lambda}^d(p'_i, p'_j, p'_k) &= i\mathcal{A}_{\lambda'_i, \lambda'_j; \lambda'_k; \lambda}(p'_i, p'_j, p'_k), \\
i\mathcal{M}_{\lambda'_i, \lambda'_j; \lambda'_k; \lambda}^Z(p'_i, p'_j, p'_k) &= \sum_{JM} N_J D_{M\lambda'_{ji}}^{J*}(\Omega_j) \sum_{\lambda''_i \lambda''_j} \int \frac{p_j''^2 dp_j''}{(2\pi)^3} i\mathcal{T}_{\lambda'_i, \lambda'_j; \lambda''_i, \lambda''_j}^J(p''_j, M_{ij}) G_0(p''_j) i\mathcal{A}_{\lambda''_i, \lambda''_j; \lambda'_k; \lambda}^{JM}(p''_j, \Omega'_k, M_{ij}) \tag{44}
\end{aligned}$$

In this case, we do not make a partial-wave decomposition for the full process but only for the ij system.

Hence, for A , we do not apply partial-wave decomposition to the initial state, and it should be treated strictly according to its definition.

With the standard definitions $|J, \lambda; \pm\rangle = \frac{1}{\sqrt{2}}(|J, +\lambda\rangle \pm \tilde{\eta}|J, -\lambda\rangle)$, the partial-wave amplitudes with fixed parity should be

$$A_{\lambda}^{J\pm} = \frac{1}{\sqrt{2}}(A_{\lambda}^J \pm \tilde{\eta}' A_{-\lambda}^J).$$

Here, λ refers to the ij system on which we perform the partial-wave decomposition, while other helicities are omitted. The above definition differs from those with partial-wave decomposition on both the initial and final states and application of parity conservation, by an additional factor $\frac{1}{\sqrt{2}}$. To be consistent with the definition of T , we change it to

$$A_{\lambda}^{J\pm} = A_{\lambda}^J \pm \tilde{\eta}' A_{-\lambda}^J.$$

With such a definition, we have

$$M_{\lambda'}^J = \sum_{\lambda''} T_{\lambda'\lambda''}^J A_{\lambda''}^J = \frac{1}{2} \sum_P M_{\lambda'}^{JP} = \frac{1}{4} \sum_P T_{\lambda'\lambda''}^{JP} A_{\lambda''}^{JP} = \frac{1}{2} \sum_P T_{\lambda'i}^{JP} A_i^{JP} = \frac{1}{2} \sum_P T_{\lambda'}^{JP} A^{JP} \quad (45)$$

$$\mathcal{A}_{\lambda_i''\lambda_j''\lambda_k''\lambda}^{JP M}(\dots) = N_J \int d\Omega_j'' \left[D_{M,\lambda_{ji}''}^J(\phi_j'', \theta_j'', 0) \mathcal{A}_{\lambda_i''\lambda_j''\lambda_k''\lambda}(\dots, \Omega_j'', \dots) + \eta'' D_{M,-\lambda_{ji}''}^J(\phi_j'', \theta_j'', 0) \mathcal{A}_{\lambda_i''\lambda_j''\lambda_k''\lambda}(\dots, \Omega_j'', \dots) \right] \quad (46)$$

$$i\mathcal{M}_{\lambda_i'\lambda_j'\lambda_k'\lambda}^Z(p_k, p_i, p_j) = \frac{1}{2} \sum_{JPM} N_J D_{M\lambda_{ji}'}^{J*}(\Omega_j') \int \frac{p_j'^2 dp_j''}{(2\pi)^3} \sum_{i''j''} i\mathcal{T}_{\lambda_i'\lambda_j';i''j''}^{JP}(\mathbf{p}_j', M_{ij}) G_0(\mathbf{p}_j'') i\mathcal{A}_{\lambda_i''\lambda_j''\lambda_k'\lambda}^{JP M}(\mathbf{p}_j'', \Omega_k', M_{ij}) \quad (47)$$

The amplitude are calculated in [qBSE.TGA](#).

Note that in most cases, other mechanisms such as background must be considered, and hence the above treatment cannot be applied. Instead, the amplitudes should be calculated directly using

$i\mathcal{M}_{\lambda_i'\lambda_j'\lambda_k'\lambda}^Z(p_k, p_i, p_j)$, where the indices between T and A involve only the independent helicities.

However, for the final state, all helicities must be summed; therefore, we need to use the relation between the physical helicities and the independent helicities.

If A is a $2 \rightarrow 2$ process, the partial-wave decomposition can be applied to both the initial and final states.

$$\begin{aligned} i\mathcal{M}_{\lambda_1'\lambda_2'\lambda_1\lambda_2}^Z(p_1', p_2', p_1, p_2) &= \sum_{\lambda_1'\lambda_2'} \int \frac{p_2'^2 dp_2'' d\Omega_2''}{(2\pi)^3} \sum_{J'M'} N_{J'}^2 D_{M'\lambda_{21}}^{J'*}(\Omega_2') i\mathcal{T}_{\lambda_1'\lambda_2';\lambda_1'\lambda_2'}^{J'M'}(\mathbf{p}_2'', \dots) D_{M'\lambda_{21}}^{J'}(\Omega_2'') \\ &\quad \cdot G_0(\mathbf{p}_2'') \sum_{JM} N_J^2 D_{M\lambda_{21}}^{J*}(\Omega_2'') i\mathcal{A}_{\lambda_1''\lambda_2''\lambda_1\lambda_2}^{JM}(\mathbf{p}_2, \dots) \delta_{M\lambda_{21}} \\ &= \sum_{\lambda_1'\lambda_2'} \int \frac{p_2'^2 dp_2'' d\Omega_2''}{(2\pi)^3} \sum_J N_J^2 D_{\lambda_{12}\lambda_{21}}^{J*}(\Omega_2') i\mathcal{T}_{\lambda_1'\lambda_2';\lambda_1'\lambda_2'}^J(\mathbf{p}_2'', \dots) G_0(\mathbf{p}_2'') i\mathcal{A}_{\lambda_1''\lambda_2''\lambda_1\lambda_2}^J(\mathbf{p}_2, \dots) \\ &= \sum_J N_J^2 D_{\lambda_{12}\lambda_{21}}^{J*}(\Omega_2') \sum_{\lambda_1''\lambda_2''} \int \frac{p_2'^2 dp_2''}{(2\pi)^3} i\mathcal{T}_{\lambda_1'\lambda_2';\lambda_1''\lambda_2''}^J(\mathbf{p}_2'', \dots) G_0(\mathbf{p}_2'') i\mathcal{A}_{\lambda_1''\lambda_2''\lambda_1\lambda_2}^J(\mathbf{p}_2'', \dots) \\ &\equiv \sum_J N_J^2 D_{\lambda_{12}\lambda_{21}}^{J*}(\Omega_2') i\mathcal{M}_{\lambda_i'\lambda_j';\lambda_i\lambda_j}^J. \end{aligned} \quad (48)$$

$$\begin{aligned} \mathcal{A}_{\lambda_i''\lambda_j''\lambda_i\lambda_j}^{JP} &= 2\pi \int d\cos\theta'' \left[d_{\lambda,\lambda''}^J(\theta'') \mathcal{A}_{\lambda_i''\lambda_j''\lambda_i\lambda_j} + \eta'' d_{\lambda,-\lambda''}^J(\theta'') \mathcal{A}_{-\lambda_i'',-\lambda_j''\lambda_i\lambda_j} \right] \\ &= 2\pi \int d\cos\theta'' \left[d_{\lambda\lambda''}^J(\theta'') \mathcal{A}_{\lambda_i''\lambda_j''\lambda_i\lambda_j} + \eta d_{-\lambda,\lambda''}^J(\theta'') \mathcal{A}_{\lambda_i''\lambda_j'',-\lambda_i,-\lambda_j} \right], \end{aligned} \quad (49)$$

$$\int d\Omega' \sum_{\lambda_i'\lambda_j'\lambda_i\lambda_j} |i\mathcal{M}_{\lambda_i'\lambda_j'\lambda_i\lambda_j}^Z|^2 = \sum_{\lambda_i'\lambda_j'\lambda_i\lambda_j,J} N_J^2 |i\mathcal{M}_{\lambda_i'\lambda_j'\lambda_i\lambda_j}^J|^2 = \sum_{ij,J} N_J^2 |i\mathcal{M}_{ij}^{JP}|^2. \quad (50)$$

$$i\mathcal{M}_{ij}^{JP} = \int \frac{p_j'^2 dp_j'}{(2\pi)^3} \sum_k i\mathcal{T}_{ik}^{JP} G_0 i\mathcal{A}_{kj}^{JP}$$

qBSE package

The qBSE package is used to solve the Bethe-Salpeter equation with some auxiliary functions.

Data Structures for the Interactions

In the qBSE package, interaction and system information are encapsulated in dedicated data structures to facilitate efficient computation and clear organization.

structSys

The `struct structSys` (often referenced as `SYS` in the code) stores information about the system and the generally used discretization and angular integration data. Its fields include:

- `Sys::String`: A label identifying the system.
- `kv::Vector{Float64}`, `wv::Vector{Float64}`: Discretized momentum points and weights for the momentum discretization (used in numerical integration).
- `xv::Vector{Float64}`, `wxv::Vector{Float64}`: Discretized values and weights of $\cos \theta$.
- `d::Vector{Matrix{Float64}}`: Precomputed Wigner d -matrices of θ .
- `pv::Vector{Float64}`, `wpv::Vector{Float64}`: Discretized azimuthal angles and weights of ϕ discretization.
- `sp::Vector{Float64}`, `cp::Vector{Float64}`: Sine and cosine values of the discretized ϕ angles.
- `expphi::Matrix{Complex{Float64}}`: Precomputed $e^{i\phi}$ factors for partial-wave projections.
- `ChUA::Symbol`: Selects the chiral unitary approach variant; `:off` disables it, `:qBSE` uses the qBSE propagator, and `:oset1405` uses the standard oset1405 prescription.
- `potential::Symbol`: Specifies whether the potential is `:PW` or `:nopW`.
- `cutoff_type::Symbol`: Defines the cutoff scheme: `:infy` for an infinite cutoff with exponential form factor, or `:cut` for a finite momentum cutoff.
- `cutoff_re_type::Symbol`: Type of cutoff applied to the constituent (rearranged) particles. Options include `:Lambda` (fixed Λ), `:alpha` ($\Lambda = m + 0.22\alpha$, with m the exchanged meson mass), and `:alpha_light` (uses the mass of the light meson).
- `cutoff_ex_type::Symbol`: Type of cutoff for the exchanged meson (e.g., `:Lambda` for a fixed value).
- `cutoff_ex::Float64`: Numerical value of the cutoff for the exchanged meson.
- `FF_ex_type::Int64`: Integer flag indicating the form-factor type for the exchanged meson (e.g., `3` for a dipole form).
- `channel::Dict{String, Int64}`: A dictionary storing the mapping between particle pairs and their corresponding channel number.

structInterAction

The `struct structInterAction` structure (typically used as `IA` in the code) stores the properties of each interaction or exchange process. Its fields are:

- `Nex::Int64`: Total number of exchange particles or processes.
- `key_ex::Vector{String}`: Labels of the exchanged particles or interactions.
- `J_ex::Vector{Int64}`, `Jh_ex::Vector{Int64}`, `P_ex::Vector{Int64}`, `m_ex::Vector{Float64}`: Spin, parity, and mass of exchanged particles.
- `dc::Vector{Int64}`: Indicates whether the exchange is direct or crossed.
- `Ff::Vector{Float64}`: Flavor factors associated with each exchange.

These definitions ensure that all relevant quantum numbers and parameters for each interaction are explicitly tracked, supporting flexible and accurate construction of the interaction kernel in qBSE calculations.

Data Structures for the Dimensions

In the qBSE approach, matrix dimensions for a coupled-channel system are organized hierarchically: the outermost level corresponds to different channels, the next to independent helicities, and the innermost to discretized momentum points. The data structures are designed to clearly separate and manage these levels.

structChannel

The `struct structChannel` structure encapsulates the properties of a physical channel in the qBSE package, often used as `CH` in the code. Its fields include:

- `p::Tuple{String,String}, p_name0::Tuple{String,String}, Tuple{Int,Int}, m::Tuple{Float64,Float64}, Tuple{Int64,Int64}, Jh::Tuple{Int64,Int64}, P::Tuple{Int64,Int64}`: Vectors specifying the key of particle p and those without charge `p_name0`, label for antiparticle, mass m , spin $J=J/Jh$, and parity (P) for each particle in the channel. Here, $Jh = 1$ for integer spin and $Jh = 2$ for half-integer spin.
- `cutoff::Float64` specifying the cutoff parameter for the channel.
- `IHb::Int64, IHe::Int64, IHn::Int64` indicating the starting and ending indices of independent helicities, and the total number of independent helicity states for the channel.

These definitions ensure that each channel's quantum numbers and relevant parameters are explicitly tracked, supporting efficient matrix construction and manipulation in qBSE calculations.

structIndependentHelicity

The `mutable struct structIndependentHelicity` defines the independent helicity states in the qBSE package, commonly referred to as `IH` in the code. Its fields are:

- `iCH::Int64`: Index of the channel to which this independent helicity belongs.
- `hel::Tuple{Int64,Int64}, helh::Tuple{Int64,Int64}`: Tuples containing the helicity values and corresponding fermion/boson indicators for the independent helicity states, respectively.
- `Dimb::Int64, Dime::Int64`: The starting and ending indices, defining the dimension range for this helicity state.
- `k::ComplexF64, w::Float64`: The discretized momentum value and its associated integration weight, used in numerical quadrature.

These fields facilitate the indexing and dimensionality management of helicities, particularly in handling both on-shell and off-shell contributions within the qBSE framework.

`Dim::Matrix{Int64, 3, N}` and `Dimt::Array{Int64, 3, NE}`

The `Dim` matrix stores three pieces of information for each dimension in the total matrix:

- Element 1: Index of the independent helicity to which this dimension belongs.
- Element 2: Sequential order of this dimension within its corresponding helicity.
- Element 3: Indicator for an extra on-shell dimension (+1 if $W > \sum m$, otherwise 0).

Here, `N` denotes the total number of dimensions, and `NE` represents the number of energy points. The `Dimt` array extends this structure across multiple energy points.

These auxiliary data structures ensure proper association between each discretized momentum point and its corresponding helicity and integration weight, thereby enabling accurate matrix construction and numerical computations in the qBSE framework.

Additional Data Structures

`structMomentum`

The `struct structMomentum` structure stores the four-momenta and related kinematic quantities for a $2 \rightarrow 2$ scattering process. Its fields are:

- `i1::SVector{5,ComplexF64}, f1::SVector{5,ComplexF64}`: Initial and final momentum of particle 1.
- `i2::SVector{5,ComplexF64}, f2::SVector{5,ComplexF64}`: Initial and final momentum of particle 2.
- `q::SVector{5,ComplexF64}`: Momentum of the exchanged particle.
- `q2::Complex{Float64}`: Invariant mass squared (q^2) of the exchange.
- `qt::Complex{Float64}`: Additional kinematic variable (e.g., t -channel momentum transfer).

`structHelicity`

The `mutable struct structHelicity` structure stores the helicity information for the particles in a $2 \rightarrow 2$ interaction. Its fields are:

- `i1::Int64, i1h::Int64`: Helicity and denominator for initial particle 1.
- `f1::Int64, f1h::Int64`: Helicity and denominator for final particle 1.
- `i2::Int64, i2h::Int64`: Helicity and denominator for initial particle 2.
- `f2::Int64, f2h::Int64`: Helicity and denominator for final particle 2.

Particle

`particles!(filename::String)`

Reads particle information from a formatted data file, dynamically generates a concrete type, and injects the result into the module-level `const qBSE.p`.

Arguments

- `filename::String` — Path to the particle data file.

Behavior

1. Reads the first line of the file as the **header**. Each header field becomes a field name of the generated type.
2. Infers the type of each column from the first data row:
 - `tryparse(Int, v)` succeeds for the whole column $\rightarrow$ `Int`
 - otherwise `tryparse(Float64, v)` succeeds $\rightarrow$ `Float64`
 - otherwise $\rightarrow$ `String`

3. Uses `Core.eval` to generate a concrete `struct` type `T` in the current module. Field names come from the header; field types come from step 2.
4. Parses each remaining data row, constructs a `T` instance, and stores it in a `Dict{String, T}` keyed by the first column (the charged-particle name).
5. Binds that dictionary as the module-level `const p` via `Core.eval`. If `p` already exists, an error is raised telling the user to restart the Julia session.

Returns

- `Dict{String, T}` — The newly constructed particle dictionary. Side effect: the module-level `qBSE.p` is bound to the same dictionary.

Usage

```
@everywhere qBSE.particles!("particles.txt")
```

Afterwards, access it from inside the package or from the main program:

```
qBSE.p["11"].m      # ::Float64
qBSE.p["11"].J      # ::Int
qBSE.p["11"].nameL  # ::String
```

Notes

1. **Call only once.** `p` is `const`; calling `particles!` again raises an error. Restart the session to switch files.
2. Under `@everywhere`, each worker generates its own type and `p` independently.
3. **The first data row must exist and follow the strict format.** Whether a column is `Float64` or `Int` depends on the distinction between `1.0` and `1`, so the data file must respect this convention.
4. **Do not call `particles!` at package top level or in a `const` initializer.** Otherwise `Core.eval` runs during precompilation and breaks it.
5. `Base.invokeatest` is used for `Dict{String, T}()` and `T(vals...)`. The type is generated at runtime.
6. **Accessing `p` inside the package:** you can write `p[key].field` directly, because `p` has already been injected into the module by `Core.eval` and is present when later code is compiled. If a function is compiled before injection but executed after, use `getfield(@__MODULE__, :p)` to resolve the binding at runtime.

Functions for the qBSE

function preprocessing(Sys, qn, channels, Ff, config, Np, Nx, Nphi)

This function is designed to be called within `res` to prepare the system and channel data structures for qBSE calculations, including the necessary discretization and quantum number information.

Arguments:

- `Sys::String`: Label identifying the system, stored in `SYS.Sys`.
- `qn`: Quantum numbers for the process, and labels for Riemann sheets.
- `channels`: List of channels to be included in the calculation, stored in `IA[]`.

- **Ff**: Flavor factors, stored in **IA[]**.
- **Np, Nx, Nphi**: Number of momentum discretization points, $\cos \theta$ discretization points, azimuthal angle discretization points.
- **config**: A **NamedTuple** (or **Dict**) providing runtime configuration flags. The user may supply only the required fields; omitted fields are set to their defaults as listed below:
 - **ChUA = :off** – Selects the chiral unitary approach (cutoff regularization only; dimensional regularization planned):
 - **:off** – standard qBSE (no ChUA).
 - **:qBSE** – ChUA using the qBSE propagator.
 - **:oset1405** – standard ChUA used in the calculation of $\Lambda(1405)$.
 - **:oset980** – standard ChUA used in the calculation of $f_0(980)$.
 - **cutoff_type = :infty** – Defines the ultraviolet regularization scheme:
 - **:infty** – infinite momentum cutoff with an exponential form factor (form factor provides natural damping). **Note**: **potential** must be **:nopW** in this case.
 - **:cut** – finite hard momentum cutoff (integration upper limit $=q_{max}$). **Note**: **cutoff_re_type** should be set to **:Lambda** when using this scheme.
 - **potential = :nopW** – Specifies how the potential is treated:
 - **:PW** – potential after partial-wave decomposition.
 - **:nopW** – bare potential without partial-wave decomposition.
 - **cutoff_re_type = :Lambda** – Type of cutoff applied to constituent (rearranged) particles:
 - **:Lambda** – fixed constant Λ .
 - **:alpha** – $\Lambda = m_{ex} + 0.22 \cdot \alpha$, where m_{ex} is the mass of the exchanged meson and α is a tunable parameter.
 - **:alpha_light** – uses the mass of the lightest meson (e.g., pion) as a reference.
 - **cutoff_ex_type = :Lambda cutoff_ex = 0.0 FF_ex_type = 3** see function **propFFex()**.

Important constraints:

- When **cutoff_type = :infty**, **potential** must be **:nopW**.
- When **cutoff_type = :cut**, **cutoff_re_type** should be **:Lambda** (fixed cutoff).
- The **ChUA = :qBSE, :oset1405** or **:qBSE** options are currently **implemented only for cutoff_type = :cut**; support for **:infty** is **not yet implemented** and will be added in a future release. Using these options with **:infty** may lead to incorrect results or runtime errors.

Example for arguments:

```
if Sys == "KNcp"
  qn = (I=1, Ih=1, J=1, Jh=2, P=-1, C=-1, lRm=1)
  Range = (ERmin=1.2, ERmax=2.0, NER=200, EIt=0.200, NEI=20, Ep=("cm",))
  cutoff = (cutoff_re_type=:alpha_light,)
  channels = (
    ("K_b0:N_p", 1.63),
    ("pi_0:Sigma_p", 1.63),
    ("pi_p:Sigma_0", 1.63),
    ("pi_p:Lambda", 1.63),
    ("eta:Sigma_p", 1.63)
  )
  Ff = Dict{String,Any}{
    "K_b0:N_p->K_b0:N_p" => ([[ "V", 1], 1.0],),
    "K_b0:N_p->pi_0:Sigma_p" => ([[ "V", 1], -sqrt(0.5)],),
    "K_b0:N_p->pi_p:Sigma_0" => ([[ "V", 1], sqrt(0.5)],),
```

```

"K_b0:N_p-->pi_p:Lambda" => ([[ "V", 1], -sqrt(1.5)],),
"K_b0:N_p-->eta:Sigma_p" => ([[ "V", 1], sqrt(1.5)],),
"pi_0:Sigma_p-->pi_p:Sigma_0" => ([[ "V", 1], -2.0],)
)
end

```

Returns:

- **SYS::structSys**: The processed system object containing discretization and system information.
- **IA::Matrix{structInterAction}**: The processed interaction objects, organized as a matrix.
- **CH::Vector{structChannel}**: The processed channel objects, each describing a physical channel.
- **IH::Vector{structIndependentHelicity}**: The processed independent helicity objects, ready for use in qBSE calculations.

**function FFre(k, cutoffi, cutofff;
cutoff_re_type=:Lambda, CHi=nothing,
CHf=nothing, key_ex=0)**

In the **fV** function, form factors for constituent partilces can be included via the auxiliary function **FFre** for the regularization, which is used to regulate the interaction kernel.

Arguments:

- **k::structMomentum**: The kinematic information for the process.
- **cutoffi::Float64, cutofff::Float64**: Cutoff parameters for the initial and final particles, typically set as **cutoffi = CHi.cutoff, cutofff = CHf.cutoff**.
- **cutoff_re_type**: The type of cutoff used for the exponential regularization of the constituent particles. Use **:Lambda** for a fixed Λ , **:alpha** for $\Lambda = m + 0.22\alpha$, where m is the mass of the exchanged meson, and **:alpha_light** for using the mass of the light meson.
- **CHi, CHf, ex::Int64**: Used for the type **alpha_light** and **alpha**, respectively.
- **key_ex::Int64**: Index of the exchanged particle, usually set as **key_ex = IA0.key_ex[le]**, where **le** runs over the exchanged mesons in **1:IA0.Nex**.

Description:

These options allow flexible control over the inclusion and type of form factors in the potential kernel, supporting different regularization schemes as needed for the physical system under study.

**function propFFex(k, key_ex, cutoff;
cutoff_ex_type=:Lambda, FF_ex_type=3)**

In the **fV** function, form factors for the exchanged mesons can be included via the auxiliary function **propFFex**.

Arguments:

- **k::structMomentum**: The kinematic information for the process.
- **ex::Int64**: Index of the exchanged particle, usually set as **ex = IA0.key_ex[le]**, where **le** runs over the exchanged mesons in **1:IA0.Nex**.
- **cutoff::Float64**: The cutoff parameter for the exchanged meson, often chosen as the same as the regulations.
- **cutoff_ex_type**: The type of cutoff used for the exchanged mesons. Use **:Lambda** for a fixed Λ , **:alpha** for $\Lambda = m + 0.22\alpha$, where m is the mass of the exchanged meson.
- **FF_ex_type::Int64**: Type of form factor for the exchanged meson:

- 0: No form factor.
- 1: $\frac{L^2 - m^2}{L^2 - q^2}$
- 2: $\frac{L^4}{(m^2 - q^2)^2 + L^4}$
- 3: $\exp\left(-\frac{(m^2 - q^2)^2}{L^4}\right)$
- 4: $\frac{L^4 + (q_t - m^2)^2/4}{(q^2 - (q_t + m^2)/2)^2 + L^4}$
- 5: $\frac{L^2}{L^2 - q^2}$
- 6: $\exp\left(-\frac{(m^2 - q^2)^2}{2L^4}\right)$

Description:

These options allow flexible control over the inclusion and type of form factors in the potential kernel, supporting different regularization schemes as needed for the physical system under study.

function fV(k, l, SYS, IA0, CHf, CHi, VVertex)

This function defines the potential kernel for qBSE calculations by extracting vertices or the direct potential from VVertex, which is then passed to the kernel function. It is responsible for computing the interaction potential between particles in a scattering process.

Arguments:

- **k::structMomentum, l::structHelicity**: The momenta and helicities of initial and final particles.
- **SYS::structSys**: The same as above.
- **IA0::structInterAction**: The interaction object containing information about the interactions and their properties, one of element of **IA** above.
- **CHf::structChannel, CHi::structChannel**: The final and initial channel object containing information about the final state particles, one of element of **CH** above.
- **VVertex**: The vertices of the interactions or direct potential for qBSE, which should be defined in main file.

Returns:

A value representing the potential kernel, suitable for use in the qBSE package's calculations.

Note: This is an internal function intended for use within the qBSE package. Users should not call this function directly.

Functions for rescattering amplitudes and poles

auxiliary function

qBSE.showSYSInfo(Range, qn, IA, CH, IH)

This function is used to display the system information, including the range of energies, quantum numbers, interaction information, and channel information.

qBSE.showPoleInfo(qn, Ec, reslog, "data/output.txt")

This function is used to display the pole information, including the energy, width, and other relevant parameters. The results are saved in a file named "output.txt" in the "data" directory.

function resc0(Range, iER, qn, SYS, IA, CH, IH, VVertex)

This function is designed to be called within `res` to calculates the rescattering process by qBSE for a given range of energies.

Arguments:

- **Range**: The range of energies to be considered. For example `Range = (ERmin=1.2, ERmax=2.0, NER=200, EIt=0.200, NEI=20, Ep="cm")`
- **iER**: The index of the energy range, energy can be obtained as $ER = Range.ERmax - iER * (Range.ERmax - Range.ERmin) / (Range.NER - 1)$.
- **qn**: The quantum numbers for the process.
- **SYS, IA, CH, IH**: Obtained by `preprocessing`.
- **VVertex**: Interaction vertices or direct potentials for qBSE framework. Flow: `VVertex` $\rightarrow$ `res` $\rightarrow$ `VGI()` $\rightarrow$ `kernel()` $\rightarrow$ `fV()` in computation pipeline.
- **eps**: Optional regularization parameter (default: 1e-4im) to add a small imaginary part to propagator denominators to avoid singularities.

Returns:

- **Ect::Vector{ComplexF64}**: The complex energy values for the system.
- **reslogt::Vector{Float64}**: The values of $\log |1 - VG|$.
- **resM2::Matrix{Float64}**: The matrix $|M|^2$ for the channels.
- **IH::Vector{structIndependentHelicity}**: The independent helicity objects, recalculated from the input `IH`.
- **Dim::Vector{structDimension}**: The dimension objects, recalculated from the input `Dim`.
- **TG::Matrix{ComplexF64}**: The TG matrix used for decays.

function resc(Sys, qn, Range, channels, Ff, cutoff, VVertex; Np=10, Nx=10, Nphi=5)

This function, which employs parallel computation to model the rescattering process via the qBSE (quantum Bethe-Salpeter equation), is designed to be called from the main program. The core calculation is implemented in the function `res0`.

Arguments:

- **Sys::String**: Label identifying the system, stored in `SYS.Sys`.
- **qn**: Quantum numbers for the process, and labels for Riemann sheets.
- **Range**: The range of energies to be considered. For example `Range = (ERmin=1.2, ERmax=2.0, NER=200, EIt=0.200, NEI=20, Ep="cm")`
- **channels**: List of channels to be included in the calculation, stored in `IA[]`.
- **Ff**: Flavor factors, stored in `IA[]`.
- **cutoff**: A `NamedTuple` for the last four keys of `structSys`. The user may provide only the required fields; the omitted ones will be set to default as `cutoff = (cutoff_re_type = :Lambda, cutoff_ex = 0.0, cutoff_ex_type = :Lambda, FF_ex_type = 3)`.
- **Np, Nx, Nphi**: Number of momentum discretization points, $\cos \theta$ discretization points, azimuthal angle discretization points.

Returns:

- **Ect::Vector{ComplexF64}**: The complex energy values for the system.
- **reslogt::Vector{Float64}**: The values of $\log |1 - VG|$.
- **resM2::Matrix{Float64}**: The matrix $|M|^2$ for the channels.

- `IH::Vector{structIndependentHelicity}`: The independent helicity objects, recalculated from the input `IH`.
- `Dim::Vector{structDimension}`: The dimension objects, recalculated from the input `Dim`.
- `TG::Matrix{ComplexF64}`: The TG matrix used for decays.

function simpleXsection(ER, resM2, CH, qn; Ep="cm",)

This function computes the total cross section for a $2 \rightarrow 2$ scattering process using the squared amplitude matrix and channel information.

Arguments:

- `ER::Vector{Float64}`: Array of energy values (typically from `Ect`).
- `resM2::Matrix{Float64}`: Matrix of squared amplitudes $|M|^2$ for the process, as obtained from the qBSE calculation.
- `CH`: Vector of channel structures, as described above.
- `qn`: Quantum numbers for the process.
- `Ep`: Energy frame to use (`("cm", )` for center-of-mass, `("L", p1, p2)` for laboratory; default: `"cm"`).

Returns: A `Vector{Matrix{Float64}}` value representing the total cross section at each energy point.

Description: The function calculates the cross section by summing or integrating over the relevant momentum discretization points and applying the appropriate kinematic and symmetry factors, including spin averaging and flux normalization. It uses the channel and quantum number information to determine the initial and final state properties.

function lambda(m1, m2, m3)

This function calculates the Kallen function $\lambda(m_1, m_2, m_3)$, which is used in the phase space integration.

Decay

function proc(pf, pin, amps)

This function constructs a channel tuple for use in decay width or cross section calculations, such as with `Xs.Xsection`.

Arguments:

- `pf`: List of final state particle names (e.g., `["pi_m", "pi_p", "pi_p", "Lambda"]`).
- `pin`: List of initial state particle names (can be empty if not needed).
- `amps`: Amplitude information or function.

Returns:

A tuple of the form `(pf=pf, namef=namef, mf=mf, pin=pin, namei=namei, mi=mi, amps=amps, ranges=ranges)`, where:

- `pf, pin`: Final and initial state particle names.
- `namef, namei`: Channel names for final and initial states.
- `mf, mi`: Masses of final and initial state particles.
- `amps`: Amplitude information.

- **ranges**: Specifies spin value ranges for final state particles. Example: (0:0, -1:1, -1:2:1) represents spins for π , ρ , and N in a $\pi\rho N$ system.

Example:

To compute the decay width for $\Lambda_c \rightarrow \pi^- \pi^- \pi^+ \Lambda$:

```
ch = qBSE.ch(["pi_m", "pi_p", "pi_p", "Lambda"], [], amps)
```

The resulting **ch** can be passed directly to **Xs.Xsection**. See the **Xs.Xsection** documentation for further usage details.

function LorentzBoost(k::SVector{5,Float64}, p::SVector{5,Float64})

The function **LorentzBoost** takes two arguments: **k**, which is a 5-component momentum vector, and **p**, which is a 4-component momentum vector. The function performs a Lorentz boost on the momentum vector **k** using the momentum vector **p**.

```
function LorentzBoost(momenta::Vector{SVector{5,Float64}},
p::SVector{5,Float64})
```

Here the **momenta** is a vector of 5-component momentum vectors. The function applies the Lorentz boost to each momentum vector in the array **momenta** using the momentum vector **p**.

function setTGA(par, sij, k, tecm, i, j)

set the frame and other things for calculating TGA, which should be used before **TGA** function.

Arguments:

- **par**: The parameters for the TGA calculation.
- **k**: The momentum of the particle in the center-of-mass (CM) frame.
- **tecm**: The total energy in the center-of-mass frame.
- **i**: The index of the first particle.
- **j**: The index of the second particle.

Returns:

```
para=(E=sqrt(sij), par=par, IH=IH, Dim=Dim, k=kn, P=Pn,resc=(i,j))
```

TGA(para, cfinal, cinter, ranges)

This function calculates the transition amplitude for a given final state **cfinal** and interaction **cinter**.

Arguments:

- **para**: The calculation parameters, as returned by **setTGA**.
- **cfinal::String**: The final state configuration (e.g., String with ":" for the outgoing particles).
- **cinter::Tuple{String, Float64, Function, NamedTuple}**: The indices of intermediate channels (**ch::String**) with weights (**weight::Float64**), the corresponding vertex functions (**Vertex::Function**), and associated data, including the vertex structure, precomputed spinors or polarization vectors (**cached::NamedTuple** e.g., **cached=(ULc=GA1 * FR.U(para14.P, lLc),)**). The content in **cached** will be used in functions **Vertex**, such as **Vertex14(k, P, l,**

`cached) = FR.U(k[4], l[4], bar=true) * cached.ULc`, defined in `function amps` in main file.

function Vertex14(k, P, l, Vert)

This function computes the initial decay vertex for a process involving two rescattering particles.

Arguments:

- **k**: Momenta of the two rescattering particles and all other final-state particles.
- **P**: Four-momentum of the parent (initial) particle.
- **l**: Helicity indices of the two rescattering particles and all other final-state particles.
- **Vert**: Vertex structure or function, defined in `function TGA`

Returns:

A value representing the decay vertex amplitude, suitable for use in the qBSE package's transition amplitude calculations.